Form Handling with PHP & MySQL

Eng. Eric Byiringiro

Instructor, Web Design & Programming

HYPERTEXT PREPROCESSOR (PHP)

- The PHP Hypertext Preprocessor (PHP) is a programming language that allows web developers to create dynamic content that interacts with databases. PHP is basically used for developing web based software applications.
- **What is PHP?**

✓ PHP stands for: **H**ypertext **P**reprocessor
✓ PHP is a server-side scripting language
✓ PHP scripts are executed on the server
✓ PHP supports many databases (MySQL,Oracle, PostgreSQL, Generic ODBC, etc.)
✓ PHP files can contain text, HTML tags and scripts
✓ PHP files are returned to the browser as plain HTML
✓ PHP files have a file extension of ".php", ".php3", or ".phtml"

What Can PHP Do?

- PHP can generate dynamic page content
- PHP can **create, open, read, write, delete, and close** files on the server
- PHP can collect form data
- PHP can **add, delete, modify data** in your database
- PHP can **restrict users to access some pages** on your website

❑ ***To use a web server with PHP support, you can either:***

❑ Install a local server environment like Apache (or IIS), along with PHP and MySQL; or

❑ Choose a web hosting provider that offers PHP and MySQL support.

Step-by-Step Guide to Run PHP in VS Code

- Install PHP and Add it to **C:\php** to your System PATH(edit the system environment).
- Install VS Code Extensions
- In VS Code, go to Extensions (Ctrl+Shift+X) and install:
  - **PHP Extension Pack** or
  - **PHP Intelephense** (for code intelligence)
  - PHP Server by *brapifra*
- Create PHP FileOpen VS CodeCreate a folder (e.g., php-test)Inside that folder, create a file index.php:

Basic PHP Syntax

- PHP code is executed on the server, and the plain HTML result is sent to the browser.
- A PHP scripting block always starts with **<?php** and ends with **?>**.
- A PHP scripting block can be placed anywhere in the document.
- On servers with shorthand support enabled you can start a scripting block with <? and end with ?>.
- For maximum compatibility, we recommend that you use the standard form (<?php) rather than the shorthand form.

```
<?php
?>
```

- A PHP file normally contains HTML tags, just like an HTML file, and some PHP scripting code.

Basic PHP Syntax

- <html>
- <body>
- <?php
- echo "Hello World";
- ?>
- </body>
- </html>
- Each code line in PHP must end with a semicolon. The semicolon is a separator and is used to distinguish one set of instructions from another.
- There are two basic statements to output text with PHP: **echo** and **print**. In the example above we have used the echo statement to output the text "Hello World".
- **Note:** The file must have a .php extension. If the file has a .html extension, the PHP code will not be executed.

Variables in PHP

- Variables are used for storing values, like text strings, numbers or arrays.
- When a variable is declared, it can be used over and over again in your script.
- All variables in PHP start with a $ sign symbol.
- The correct way of declaring a variable in PHP:
- $var_name = value;
- Let's try creating a variable containing a string, and a variable containing a number:

```
<?php
$txt="HelloWorld!”;
$x=16;
?>
```

Conditional Statements in PHP

1. The if...else Statement

Use the if....else statement to execute some code if a condition is true and another code if a condition is false.

Example:

The following example will output "Have a nice weekend!" if the current day is Friday, otherwise it will output "Have a nice day!":

```
 <html>
<body>
<?php
$d=date("D");
 if ($d=="Fri"){
 echo "Have a nice weekend!";
}else{
 echo "Have a nice day!";
}
 ?>
</body>
</html>
```

The PHP Switch Statement

```
<html>
<body>
<?php
    switch ($x)
{
case 1:
     echo "Number 1"; break;
case 2:
     echo "Number 2"; break;
case 3:
echo "Number 3"; break;
default:
  echo "No number between 1 and 3";
}
?>
</body>
</html>
```

PHP Loops

4. The while Loop

- The while loop executes a block of code while a condition is true.

```
<html>
<body>
  <?php
      $i=1;
       while($i<=8)          {
           echo "The number is " . $i . "<br />";
          $i++;            }
    ?>
</body>
</html>
```

PHP Loops

5. The do...while Statement

- The do...while statement will always execute the block of code once, it will then check the condition and repeat the loop while the condition is true.

```
<html>
<body>
<?php
$i=1;
do{
$i++;
echo "The number is " . $i . "<br />";
}
while ($i<=5);
?>
</body>
</html>
```

PHP Loops

- **6. The for Loop**
- The for loop is used when you know in advance how many times the script should run.

```
<html>
<body>
<?php
for ($i=1; $i<=5; $i++)
{
echo "The number is " . $i . "<br />";
}
?>
</body>
</html>
```

PHP Forms and User Input

- The most important thing to notice when dealing with HTML forms and PHP is that any form element in an HTML page will **automatically** be available to your PHP scripts.
- The PHP $_GET and $_POST variables are used **to retrieve information from forms**, like user input.

```
<html>
<body>
<form action="welcome.php" method="post">
Name: <input type="text" name="name"><br>
E-mail: <input type="text" name="email"><br>
<input type="submit">
</form>
</body></html>
```

PHP Forms and User Input

- When a user fills out the form above and click on the submit button, the form data is sent to a PHP file, called "welcome.php":
- "welcome.php" looks like this:

```
<html>
<body>
Name: <?php echo $_POST["name"]; ?><br>
Email address: <?php echo $_POST["email"]; ?>
</body>
</html>
```

The $_GET Function

- The built-in $_GET function is used to collect values from a form sent with method="get".
- Information sent from a form with the GET method is visible to everyone (it will be displayed in the browser's address bar) and has limits on the amount of information to send (max. 100 characters).
- Example:

```
<html>
<body>
<form action="welcome.php" method="get">
Name: <input type="text" name="fname" />
Age: <input type="text" name="age" />
<input type="submit" />
</form>
</body>
</html>
```

- When the user clicks the "Submit" button, the URL sent to the server could look something like this:
- http://localhost/welcome.php?fname=turinabo+sam&age=21

The $_GET Function

- The "welcome.php" file can now use the $_GET function to collect form data (the names of the form fields will automatically be the keys in the $_GET array):

```
Names: <?php echo $_GET["fname"]; ?>.<br />
Age: <?php echo $_GET["age"]; ?> years old!
```

- When using method="get" in HTML forms, all variable names and values are displayed in the URL.
- **Note:** This method should not be used when sending passwords or other sensitive information!
- However, because the variables are displayed in the URL, it is possible to bookmark the page. This can be useful in some cases.
- **Note:** The get method is not suitable for large variable values; the value cannot exceed 100 characters.

SESSION ROADMAP

What You'll Be Able to Do

1. Form submission from POST data to a saved database row.
2. Connection, binding, and execution.

The Complete Script to send data into db (myhotel)

```php
<?php
// Database connection
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "myhotel";

// Create connection
$conn = new mysqli(...);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . ...);
}
// Get form data
$name = trim($_POST['name']);
$email = trim($_POST['email']);

// Prepare SQL statement
$stmt = $conn->prepare(
    "INSERT INTO login (Name, Email)
     VALUES (?, ?)");
$stmt->bind_param("ss", $name, $email);
// Execute
if ($stmt->execute()) {
    echo "<h2>Welcome, " .
htmlspecialchars($name) . "!</h2>";
    echo "<p>Saved successfully.</p>";
} else {
    echo "Error: " . $stmt->error;
}
// Close connection
$stmt->close();
$conn->close(); ?>
```

connect → read input → prepare → bind & execute → respond → close. We'll open each block next.

From Submitted Form to Saved Row

HTML Form

User submits data via POST

→

mysqli Connect

Script opens the myhotel connection

→

Read $_POST

trim() : string fn to remove extra spacing

→

prepare()

compile a SQL statement

→

bind_param('ss')

Ensures the values are treated as the correct data type.

→

execute()

command that runs the prepared SQL statement on the database.

→

Respond

htmlspecialchars() output or error

STEP 1 · SETUP

Opening the Database Connection

```php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "myhotel";
$conn = new mysqli($servername, $username, $password,
$dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
```

- A new mysqli object opens a connection to the myhotel database on localhost.
- connect_error is checked immediately — if the connection fails, the script stops with die().
- Credentials are hard-coded here for teaching purposes only; production apps load these from environment config, not the script.

STEP 2 · INPUT

Reading the Submitted Form Data

```php
$name = trim($_POST['name']);
$email = trim($_POST['email']);
```

- $_POST holds the values submitted by an HTML <form method="post"> with fields name and email.
- trim() strips leading/trailing whitespace so " John " and "John" are treated the same.
- Nothing here checks that the fields exist or contain valid data

STEP 3 · THE KEY IDEA

Why prepare() Instead of Building a Query String?

AVOID

```
$sql = "INSERT INTO login
VALUES ('$name')"
```

Concatenating user input directly into SQL text lets an attacker inject their own SQL — e.g. entering ' ; DROP TABLE login; -- as a name.

THIS CODE

```
$stmt
= $conn->prepare(
  "INSERT INTO login (Name, Email)
   VALUES (?, ?)");
```

The ? placeholders reserve a spot for each value. The database engine compiles the query structure first, then binds data separately — user input can never change the query's meaning.

How a prepared statement flows

Query template with ? placeholders → **bind_param() attaches real values** → **execute() runs safely on the DB**

STEP 4 · BINDING & EXECUTING

Attaching Values and Running the Query

```php
$stmt = $conn->prepare(
 "INSERT INTO login (Name, Email) VALUES (?, ?)");
$stmt->bind_param("ss", $name, $email);

if ($stmt->execute()) {
    // insert succeeded
} else {
    // $stmt->error holds the failure reason
}
```

- "ss" tells MySQL both bound values are strings — one type letter per placeholder, in order (i=int, d=double, s=string, b=blob).
- bind_param() sends the values to the database separately from the query text — this is what actually blocks injection.
- execute() returns true/false, so success and failure are both handled explicitly.

STEP 5 · RESPONSE

Responding to the User Safely

```php
if ($stmt->execute()) {
    echo "<h2>Welcome, " . htmlspecialchars($name) .
"!</h2>";
    echo "<p>Your information has been saved
successfully.</p>";
} else {
    echo "Error: " . $stmt->error;
}
```

- htmlspecialchars() escapes characters like < and > in $name before it's echoed — this blocks stored/reflected XSS through the welcome message.
- The error branch echoes $stmt->error directly — fine for a classroom demo, but real apps log the detail and show the visitor a generic message.

STEP 6 · CLEANUP

Closing the Statement and Connection

```
$stmt->close();
$conn->close();
```

- close() releases the prepared statement and the database connection back to the server.
- PHP would clean these up automatically at the end of the script, but closing explicitly is good practice — especially in scripts that run many queries or loops.

Questions & Practice

Try rewriting the SQL as an unsafe, concatenated string — then explain to a partner exactly how an attacker could exploit it.